GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

P–Splines for Heteroskedastic Location–Scale Regression in R

Hausarbeit — Advanced Statistical Programming in R

by Moritz Dallmann and Nicolas Wackermann

Submitted in fulfillment of the requirements
for the degree of Master of Science (Angewandte Statistik)

15.09.2025

Study Program:	Master of Science, Angewandte Statistik
Faculty:	Faculty of Business and Economics
Chair:	Chair of Spatial Data Science and Statistical Learning
Supervisors:	Prof. Dr. Elisabeth Bergherr Quentin Seifert
Student ID:	10855073 25613208

Contents

1 Introduction **2**

2 Theoretical Background **3**
2.1 From Linear Models to Generalized Additive Models . . . 3
2.2 Polynomial Splines and B–Spline Basis Functions . . . 3
2.3 Penalized Weighted Least Squares Estimation . . . 4
2.4 Location–Scale Regression and the RS Alternation . . . 5
2.5 Smoothing Parameter Selection . . . 5

3 Usage and Implementation **7**

4 Simulation Study **11**
4.1 Introduction and experimental design . . . 11
4.1.1 Data–generating algorithms (DGAs) . . . 11
4.1.2 Methods and safety mechanisms . . . 12
4.1.3 From design to inference . . . 13
4.2 Results . . . 13
4.2.1 Mean–criterion comparison . . . 13
4.2.2 Scale criterion and predictive interval calibration . . . 15
4.2.3 Sample–size effects . . . 15
4.2.4 Basis choice and the truncated–power flag . . . 17
4.3 Practical recommendations . . . 17
4.4 Reproducibility and computational aspects . . . 18

5 Conclusion **20**

Appendix: asp25psplines manual **22**

Bibliography **33**

List of Figures **34**

1 Introduction

Many empirical data sets exhibit nonlinear signal and nonconstant variance; in such settings the assumptions of standard linear models are too restrictive. Penalized B-splines (P-splines) provide a flexible, compact representation of smooth effects while controlling roughness through regularization and accommodating systematic variability(Eilers & Marx, 1996). Extending this construction to a location–scale formulation enables the joint modeling of the conditional mean (location) and the conditional variance (scale) within one coherent framework(Rigby & Stasinopoulos, 2005; Stasinopoulos & Rigby, 2007).

This paper presents an R package called `asp25psplines` for heteroskedastic location–scale regression based on penalized (P-)splines. The implementation targets a Gaussian location–scale model with an identity link for the mean and a log link for the scale. Both components are represented by spline bases with difference-penalty regularization and are fitted by alternating penalized weighted least squares, with various key settings (basis type, knot placement) left configurable for the user to suit the analysis. Smoothing parameters for the mean and scale are selected independently on grids using standard criteria (AIC, BIC, ML, GCV)(Akaike, 1974; Schwarz, 1978; Craven & Wahba, 1979). The package provides prediction and diagnostic summaries suitable for practical analysis.

We make two contributions: (i) a robust, user-oriented implementation of Gaussian location–scale P-splines in R; and (ii) an empirical evaluation covering estimation accuracy, smoothness selection, and runtime. In addition, a simulation study benchmarks the package against , reporting like-for-like settings and metrics.

The remainder of the paper proceeds as follows. Chapter [2] develops the theoretical background: spline bases (with emphasis on B-splines and a truncated-power alternative), penalty construction, the penalized problems for the mean and the log–scale, the RS alternation, and the role of information criteria with effective degrees of freedom(Eilers & Marx, 1996; Hutchinson, 1990). Chapter [2] describes the implementation and usage of the package, including inputs, outputs, and diagnostics. Chapter [4] ; and Chapter [5] concludes.

2 Theoretical Background

Real data often depart from basic linear models through nonlinear trends and nonconstant variance; we address both with a Gaussian location–scale model whose mean and log–scale are represented by penalized B-spline smooths. We begin from a generalized additive viewpoint to justify smooth components, then specify how the smooths are represented by splines, how penalization controls their complexity, and how the two blocks (mean and log–scale) are estimated together.

2.1 From Linear Models to Generalized Additive Models

Generalized additive models (GAMs) are motivated by a common empirical problem: linear predictors are often too rigid—effects bend—and a fully nonparametric surface can be data-hungry and hard to interpret. GAMs strike a middle ground: they preserve the GLM link but let each covariate contribute through its own smooth $s_j(\cdot)$(Baayen & Linke, 2020). In this view, each s_j captures the systematic, possibly nonlinear, effect of one covariate on the transformed mean while keeping the overall predictor additive and interpretable.

$$g\big(\mathbb{E}[Y \mid \boldsymbol{z}]\big) \;=\; \eta(\boldsymbol{z}) \;=\; \beta_0 \;+\; s_1(z_1) \;+\; \cdots \;+\; s_d(z_d), \tag{1}$$

with a monotone link $g(\cdot)$ and unknown smooths $s_j(\cdot)$ estimated from the data. This lets the data determine nonlinear effects while keeping the predictor additive and interpretable. Thus two design choices remain: how to represent the smooths and how to prevent over-wiggling; we do both with B-splines and a curvature-controlling penalty(Eilers & Marx, 1996).

In our application there is one continuous covariate x and a Gaussian likelihood. We model the mean with the identity link and allow the *scale* to vary smoothly through a log link:

$$\mathbb{E}[Y \mid x] \;=\; \mu(x), \qquad \eta_\mu(x) \;=\; \mu(x), \tag{2}$$

$$\eta_\sigma(x) \;=\; \log \sigma(x). \tag{3}$$

Two smooth components—one for the mean and one for the log–scale—determine the conditional distribution. Therefore, we adopt a B–spline parameterization for both components because local support yields well-conditioned design matrices and difference penalties provide explicit roughness control; we now define the basis and its penalty, then turn to joint mean–scale estimation.

2.2 Polynomial Splines and B–Spline Basis Functions

To estimate a smooth function we approximate it by a finite basis expansion with locally supported elements placed along a knot sequence. Let $\boldsymbol{B}_\mu(x)$ and $\boldsymbol{B}_\sigma(x)$ denote spline basis *row* vectors for the mean and scale, with coefficient vectors $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$. We write

$$\eta_\mu(x) = \boldsymbol{B}_\mu(x)\,\boldsymbol{\beta}, \qquad \eta_\sigma(x) = \boldsymbol{B}_\sigma(x)\,\boldsymbol{\gamma}. \tag{4}$$

Thus, smoothing reduces to linear regression on constructed features: the design matrices $\boldsymbol{X}$ and $\boldsymbol{Z}$ collect the B-spline evaluations at the observed x_i, and the smooths are $\eta_\mu(x) = \boldsymbol{B}_\mu(x)\boldsymbol{\beta}$ and $\eta_\sigma(x) = \boldsymbol{B}_\sigma(x)\boldsymbol{\gamma}$. Thus the basis supplies the features and the coefficients do the work; what remains is to control *how much* each smooth is allowed to wiggle.

We now fix notation for B–splines so that basis construction and later penalization are unambiguous.

Formal B–spline definition (Cox–de Boor). Let $\{t_j\}$ be a non-decreasing knot sequence and $l \geq 0$ the degree. Define

$$B_{j,0}(x) = \begin{cases} 1, & t_j \leq x < t_{j+1}, \\ 0, & \text{otherwise}, \end{cases} \tag{5}$$

$$B_{j,l}(x) = \frac{x - t_j}{t_{j+l} - t_j} B_{j,l-1}(x) \; + \; \frac{t_{j+l+1} - x}{t_{j+l+1} - t_{j+1}} B_{j+1,l-1}(x), \tag{6}$$

Whenever a denominator in the Cox–de Boor fractions equals zero due to repeated knots, the corresponding term is defined to be zero; in particular, we interpret $0/0 := 0$. This makes the recurrence well defined.This convention ensures well-defined boundary behavior for B-splines.[1] B-splines are nonnegative, compactly supported, and form a partition of unity on the interior of the knot sequence—properties that yield stable, well-behaved smoothers(Lyche *et al.*, 2017). With these properties in hand, we can choose concrete bases and knots that behave well in practice.

Practical basis choices. We use cubic B–splines ($l = 3$) with m interior knots placed at empirical quantiles (here l denotes the spline degree and m the number of interior knots). Boundary knots are padded by repeating the end knots l times to stabilise edge behaviour. Knot placement can alternatively be *equidistant*; a small *boundary buffer* may extend $[a, b]$ to $[a-\delta, b+\delta]$ before placing knots to reduce edge effects. A *truncated–power* basis is also available, comprising a low-degree global polynomial part plus truncated terms $(x-\kappa)_+^q$ at knots κ. These design choices fix the feature space; in the next step we control smoothness *within* this space via penalization(Kagerer, 2013).

2.3 Penalized Weighted Least Squares Estimation

Once the basis is fixed, estimation must balance fit and smoothness. The basis representation alone does not control roughness. Penalized weighted least squares (PWLS) provides the mechanism: fit the regression while penalising curvature(Eilers & Marx, 1996). Given a current scale, the mean coefficients solve

$$\min_{\boldsymbol{\beta}} \left\| \boldsymbol{W}_\mu^{1/2} (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}) \right\|^2 \; + \; \lambda_\mu \, \boldsymbol{\beta}^\top \boldsymbol{K}_\mu \boldsymbol{\beta}, \tag{7}$$

where $\boldsymbol{y}$ is the response vector and the working weights are

$$\mathbf{W}_\mu = \operatorname{diag}\big(\exp\{-2\,\eta_\sigma(x_i)\}\big), \qquad \eta_\sigma(x) = \log \sigma(x) = \mathbf{Z}\gamma,$$

so the mean fit is weighted by $1/\sigma^2(x)$. We penalize curvature in the mean via the usual difference penalty $\mathbf{K}_\mu = \mathbf{D}_\mu^\top \mathbf{D}_\mu$, which leaves the spline's null space unpenalized. Because the weights depend on $\eta_\sigma(x)$, estimation of mean and scale is naturally linked; we return to this alternation in Section 2.4.

The penalty has the difference-penalty form $\boldsymbol{K}_\mu = \boldsymbol{D}^\top \boldsymbol{D}$, with $\boldsymbol{D}$ encoding r_{pen}-th order finite differences across adjacent coefficients; its null space corresponds to low-degree polynomials, so the penalty controls curvature without shrinking the null-space part(Eilers & Marx, 1996).

By symmetry, the scale block is updated conditionally on the mean fit. Given the updated mean, the scale coefficients solve the analogous problem for a log-scale working response $\boldsymbol{z}$:

$$\min_{\boldsymbol{\gamma}} \left\| \boldsymbol{W}_\sigma^{1/2} (\boldsymbol{z} - \boldsymbol{Z}\boldsymbol{\gamma}) \right\|^2 \; + \; \lambda_\sigma \, \boldsymbol{\gamma}^\top \boldsymbol{K}_\sigma \boldsymbol{\gamma}, \tag{8}$$

[1] See, e.g., de Boor (2001), Ch. X, on the Cox–de Boor recursion.

with the same penalty form. For the truncated-power basis, the polynomial columns are unpenalised whereas the truncated terms are penalised.

Smoother matrices and edf. To quantify model complexity under penalization, we use smoother matrices and their traces. For fixed λ_μ the penalised mean smoother is

$$\boldsymbol{S}_\mu(\lambda_\mu) = \boldsymbol{X}\big(\boldsymbol{X}^\top \boldsymbol{W}_\mu \boldsymbol{X} + \lambda_\mu \boldsymbol{K}_\mu\big)^{-1} \boldsymbol{X}^\top \boldsymbol{W}_\mu, \tag{9}$$

and the effective degrees of freedom is $\mathrm{edf}_\mu = \mathrm{tr}\big(\boldsymbol{S}_\mu(\lambda_\mu)\big)$ (analogously for the scale block). These quantities enter the selection criteria below(Eilers & Marx, 1996; Hutchinson, 1990). They provide the effective degrees of freedom terms that appear in AIC/BIC and related criteria used for tuning λ_μ and λ_σ(Eilers & Marx, 1996; Hutchinson, 1990).

2.4 Location–Scale Regression and the RS Alternation

Because the mean and scale are coupled, we fit them by *RS alternation*: condition on one block and update the other via penalized weighted least squares. Thus, following the Rigby–Stasinopoulos algorithm for generalized additive models for location, scale and shape (GAMLSS), we update one block at a time using its own working response and weights, select that block's smoothing parameter on a grid, and cycle these penalized WLS steps until the penalized likelihood (equivalently, the deviance) stabilizes (**??**). With $\boldsymbol{\mu} = \boldsymbol{X}\boldsymbol{\beta}$ and $\boldsymbol{\sigma} = \exp(\boldsymbol{Z}\boldsymbol{\gamma})$, the normal log-likelihood $\ell(\boldsymbol{\beta}, \boldsymbol{\gamma})$ defines the global deviance

$$GD(\boldsymbol{\beta}, \boldsymbol{\gamma}) \;=\; -2\,\ell(\boldsymbol{\beta}, \boldsymbol{\gamma}). \tag{10}$$

At iteration t, given $\boldsymbol{\gamma}^{(t)}$ update $\boldsymbol{\beta}^{(t+1)}$ by solving (7); then, given $\boldsymbol{\beta}^{(t+1)}$ update $\boldsymbol{\gamma}^{(t+1)}$ by solving (8). Iterate until the relative change in GD falls below a tolerance (e.g., $|GD^{(t+1)} - GD^{(t)}|/(1 + GD^{(t)}) < \varepsilon$). This RS-type scheme mirrors GAM backfitting (separate penalised smoothers) while explicitly modelling the variance through a log-scale smooth.[2] With the alternation in place, the remaining ingredient is how to *choose* the smoothing levels; we turn to this next. In practice, the sharpness of these bands depends on the smoothing parameters, which we select in the following subsection.

Prediction and visual bands. In-sample fitted values are $\hat{\boldsymbol{\mu}} = \boldsymbol{X}\hat{\boldsymbol{\beta}}$ and $\hat{\boldsymbol{\sigma}} = \exp(\boldsymbol{Z}\hat{\boldsymbol{\gamma}})$. For new covariate values x_{new}, the same knot settings (range, placement policy, degree) must be used to build $\boldsymbol{X}_{\mathrm{new}}$ and $\boldsymbol{Z}_{\mathrm{new}}$ prior to prediction. The band $\hat{\mu}(x) \pm 1.96\,\hat{\sigma}(x)$ is a *pointwise response variability band*, not a confidence band for the mean function.

2.5 Smoothing Parameter Selection

The mean and scale blocks have *independent* smoothing parameters $(\lambda_\mu, \lambda_\sigma)$. Within the RS outer cycle we select λ_μ and λ_σ independently by minimizing an information criterion computed from the corresponding blockwise fit (holding the other block fixed)(**??**); we prefer grid search for stability and reproducibility. We report $(\lambda_\mu, \lambda_\sigma)$ together with $(\mathrm{edf}_\mu, \mathrm{edf}_\sigma)$ and use coarse–to–fine grids with monotonicity checks to avoid spurious minima.

Concretely, at each outer RS iteration we evaluate a criterion on a log–spaced grid for the *active* block, keeping the other block's coefficients fixed:

$$\mathrm{AIC} = GD + 2\,\mathrm{edf}, \qquad \mathrm{BIC} = GD + (\log n)\,\mathrm{edf},$$

where GD is the global deviance and edf is the effective degrees of freedom of the active smoother (e.g., edf_μ when choosing λ_μ). While AIC/BIC with an edf correction are our defaults, maximum likelihood (ML) and generalized cross-validation (GCV) are also available

[2] See Rigby and Stasinopoulos (2005, 2007) for the GAMLSS alternating fitting scheme.

when desired(Akaike, 1974; Schwarz, 1978; Craven & Wahba, 1979). Grid search (with optional refinement around the current best value) provides reproducible choices and avoids the numerical fragility sometimes seen with gradient–based tuning.

Effective degrees of freedom. For a penalized smoother with smoother (or hat) matrix $\mathbf{S}$, the effective degrees of freedom is $\mathrm{edf} = \mathrm{tr}(\mathbf{S})$, which quantifies complexity *after* penalization. In computation, edf can be obtained exactly via matrix factorizations (e.g., Cholesky–based traces) when feasible, or approximated efficiently with a Hutchinson trace estimator when an exact evaluation would be too costly(Hutchinson, 1990). Chapter 3 details the numerical choices and safeguards we use in practice.

Summary. We have (i) framed a normal location–scale GAM with identity and log links, (ii) defined the spline bases and knot placement (with practical defaults), (iii) stated the penalized problems for mean and log–scale, (iv) described RS alternation with a relative–deviance stopping rule, and (v) outlined independent smoothing–parameter selection using AIC/BIC (with edf correction), with ML/GCV as options, including how edf is computed in practice. Chapter 3 implements these choices and discusses diagnostics.

3 Usage and Implementation

This section documents the `asp25psplines` package from both a user and an implementation perspective. The package estimates a Gaussian location–scale model, representing the conditional mean and the log–scale by smooth functions fitted via the standard RS alternation (Rigby–Stasinopoulos; see Section 3 of this chapter and (**??**)). Estimation alternates location and scale updates within a penalized weighted least squares framework. Throughout, we consider a single continuous covariate, a normal likelihood, the identity link for the mean, and the log link for the scale.

3.1 Usage

Orientation. The implementation is standalone and does not rely on external location–scale packages. A typical workflow has three steps with matching outputs: (i) prepare inputs, (ii) fit the model, and (iii) obtain fitted values and diagnostics for quick checks.

Inputs. Provide numeric vectors `x` (covariate) and `y` (response). Optional arguments control basis construction, penalties, smoothing selection, and convergence:

- **Spline size/shape:** interior knots `m`, degree `l`, difference order `r_pen`.
- **Knot placement:** `equidistant` (default) or `quantile`; optional boundary buffer to extend the range.
- **Selection criteria (per block):** `method_mu`, `method_sigma` in {`AIC`, `BIC`, `ML`, `GCV`}.
- **Smoothing grids:** `lambda_grid_mu`, `lambda_grid_sigma` (log–spaced).
- **Stopping/numerics:** `max_iterations`, `tolerance`.

These controls are set at call time. For example, one may switch between B–splines and a truncated–power basis, choose equidistant or quantile knots (with optional buffering), and select criteria and grids independently for the mean and the scale.

Fit. Call the high-level driver `run_all_funcs(x, y, ...)` to execute the alternating updates. The returned object has class `"asp25psplines_fit"` and stores `mu_hat`, `sigma_hat`, λ_μ, λ_σ, `edf_mu`, `edf_sigma`, `beta`, `gamma`, `iterations`, `x`, `y`, `X`, `Z`, `knots`, and call metadata. Separate smoothing parameters are selected for location and scale at every iteration from the specified grids using the chosen criterion.

*Quick checks.*After fitting, `summary_all(fit)` provides a compact numeric snapshot—selected λ per block, effective degrees of freedom, deviance trace, and iteration count. For a visual sanity check, `plot_all(fit)` draws the fitted mean $\hat{\mu}(x)$ and, if `show_sigma = TRUE`, the pointwise variability band $\hat{\mu}(x) \pm 1.96\,\hat{\sigma}(x)$. The legend reports λ_μ, λ_σ, edf_μ, edf_σ, and the number of iterations; axes and knots correspond to the stored design in `fit`. A typical sequence is:

```
fit <- run_all_funcs(x, y, ...)
summary_all(fit)
plot_all(fit, show_sigma = TRUE)
```

3.2 Implementation

3.2.1 Default choices

Principle. Defaults prioritize stable fits across common sample sizes and designs; users can override them when domain structure suggests otherwise:

- **Basis and knots:** $m = 20$ interior knots; degree $l = 3$ (cubic); difference order $r_{\text{pen}} = 2$; equidistant knots; boundary buffer of 5% of the data range (expands $[a, b]$ to $[a-\delta,\, b+\delta]$).

- **Smoothing grids:** λ_μ and λ_σ searched on $\exp\{\mathrm{seq}(\log 10^{-4}, \log 10^{4}, 40)\}$.
- **Selection criteria:** `method_mu` = `AIC`, `method_sigma` = `AIC` (alternatives `BIC`, `ML`, `GCV`).
- **Convergence:** `max_iterations` = 50, `tolerance` = 10^{-5} on the relative change in global deviance.
- **Numerics:** symmetric positive–definite solver with a small ridge (order 10^{-8}) and QR fallback if needed.
- **Per-block tuning:** independent smoothing grids for location and scale (λ_μ, λ_σ); criteria can differ by block.

If an optimal λ occurs at a grid boundary, expand the grid around that boundary and refit.

3.2.2 Splines

Bases. The implementation supports two bases. By default, B–splines of degree l with m interior knots are used; boundary knots are padded by repeating the end knots l times. The optional truncated–power basis consists of a global polynomial part plus truncated terms anchored at the knots.

Knot placement and buffering. By default we place m interior knots equidistant on $[a, b]$ and use cubic B–splines (degree $l = 3$). When the covariate x is strongly uneven or skewed, the option `knots = "quantile"` concentrates basis support where data are dense. To mitigate edge artefacts we extend the knot range by a small buffer δ (we use $\delta = 0.05\,(b-a)$) and pad the boundaries by repeating the end knots l times; both choices stabilize fits near the domain limits (**?**Wood, 2017). The mean and scale blocks use the same basis dimension so that penalties and edf are comparable across blocks. All knot positions, the degree, and the basis type are stored with the fit for reproducible prediction.

Penalization. Penalization follows a difference-penalty construction. For B–splines, the package builds a discrete difference matrix of order r_{pen} and sets $\boldsymbol{K} = \boldsymbol{D}^\top \boldsymbol{D}$. For the truncated–power basis, the penalty is applied to the truncated columns while leaving the polynomial part unpenalized. The same penalty form is used for both blocks, while the smoothing parameters are chosen separately. The difference penalty shrinks curvature while leaving the spline's null space (low-degree polynomials) unpenalized, which preserves broad trends and controls overfitting.

3.2.3 RS Algorithm

Goal and stop rule. The implementation alternates updates of location and scale and stops when the relative change in the global deviance falls below the tolerance. This subsection outlines the flow; full theory is given in the background chapter and is not repeated here.

Figure 1 summarizes the data flow and the alternating updates; the paragraphs below walk through each box in order.

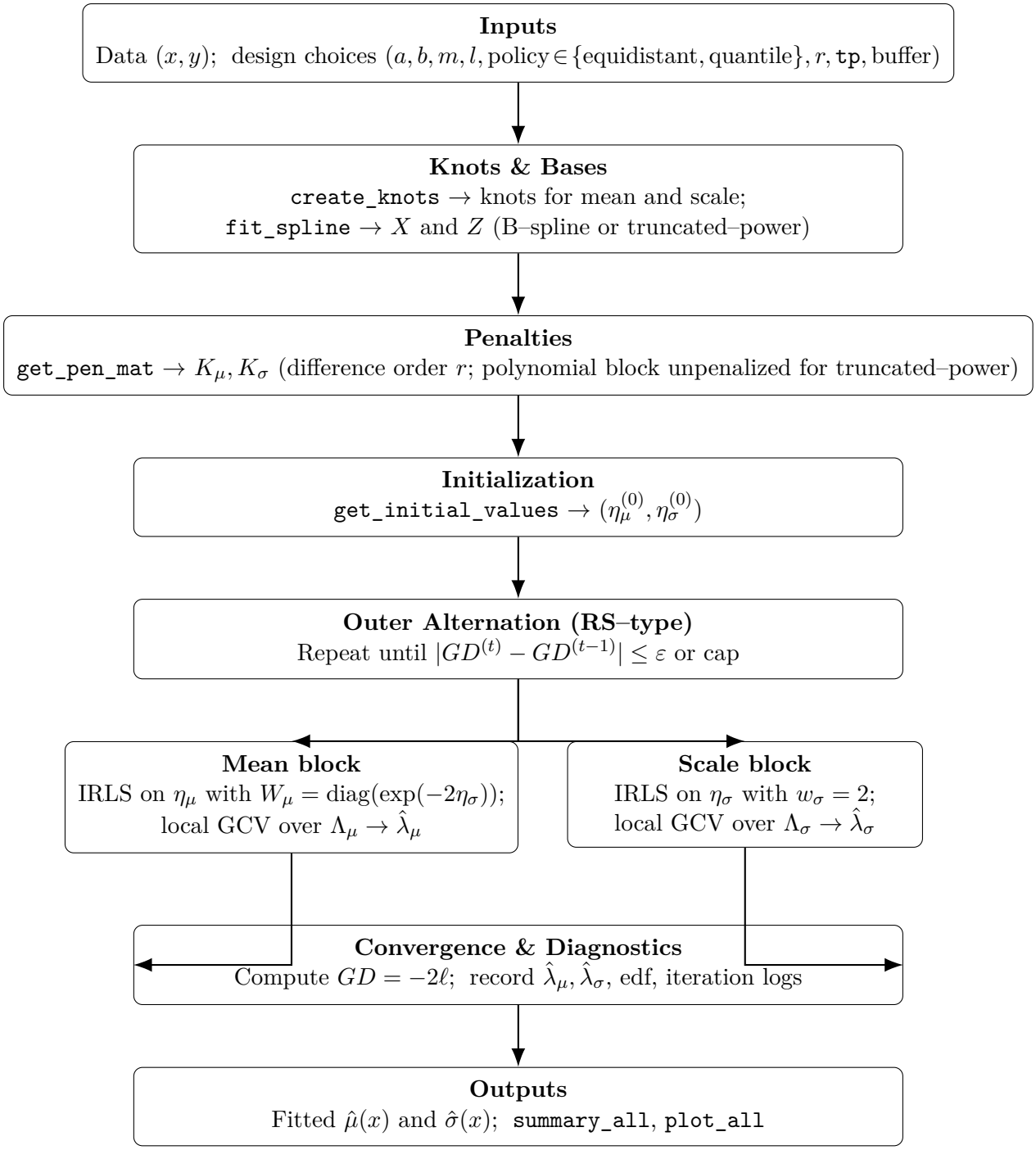

Figure 1: Flow of the standalone implementation. Penalised IRLS is used for both blocks; blockwise smoothing is selected by a criterion over a grid Λ (AIC/BIC/ML/GCV). Effective degrees of freedom are computed exactly when feasible or approximated via a Hutchinson trace estimator(Breheny, 2013; Hutchinson, 1990).

Initialization. The location block starts from a least-squares fit (with a small ridge if required). The scale block initializes from a log-residual regression to obtain a first smooth for $\log \sigma$. A small positive offset ε is added inside the logarithm to avoid $\log 0$ and is applied elementwise to residual variances; it does not affect $\hat{\sigma}(x)$ away from zero. One stabilized weighted step is taken before entering the main alternation.

Location update. Given the current scale, the mean is updated by penalized weighted least squares using the chosen basis, penalty, and a candidate λ_μ. For each λ_μ in the grid, the

code computes the deviance and an effective degrees of freedom measure from the weighted hat matrix, evaluates the selected criterion (AIC/BIC/ML/GCV), and retains the minimizer (or maximizer, for ML). The solver adds a small ridge to ensure positive definiteness and uses a QR fallback if needed(Breheny, 2013).

Scale update. Given the updated mean, the variance function is updated by smoothing a log-residual working response using the same basis and penalty structure but an independent λ_σ grid and the selected criterion. The scale update mirrors the mean update (separate grid, separate λ, same diagnostics)(Breheny, 2013).

Stopping and stored diagnostics. After each block update, the global deviance is recomputed and checked for relative change against the tolerance. The fit stores, for each iteration and block, the selected λ, the effective degrees of freedom, and the deviance trace. For the final iteration, the full criterion curve over the grid is retained to assess sensitivity (flat curves indicate near-equivalent smoothing levels). The final object exposes the standardized fields `lambda_mu`, `lambda_sigma`, `edf_mu`, `edf_sigma`, and `iterations`.

3.2.4 Numerical safeguards and EDF computation

Stabilizers. The code employs several safeguards: (i) ridge adjustments to ensure positive definiteness in linear solves; (ii) finite-value guards to prevent propagation of `NA`/`Inf` into plotting and summaries; (iii) optional time limits and retries in wrappers. Effective degrees of freedom are computed exactly via a Cholesky-based trace when feasible, and otherwise approximated efficiently with a Hutchinson trace estimator (default $n_{\text{probe}} = 15$)(Hutchinson, 1990). The λ grid is sanitized to be positive and increasing.

Notes on practical use within this flow. If the selected λ lands at a grid endpoint, expand the grid (e.g., 10^{-6}–10^{6}) around that boundary and refit. If ridge adjustments or QR fallbacks trigger frequently, consider reducing the basis dimension m or using equidistant knots. For small samples, prefer smaller m and retain the default difference order $r_{\text{pen}} = 2$.

Notes on experiments. Simulation and stress tests relying only on the public API are reported in Chapter 4; code to reproduce figures uses `run_all_funcs`, `summary_all`, and `plot_all`.

4 Simulation Study

4.1 Introduction and experimental design

The core aim of the simulation is to understand how the `asp25psplines` estimator behaves when the true mean and scale vary in complexity and when different smoothing–parameter selection rules are applied. In particular, we seek to answer four guiding questions:

1. *Mean selection.* How do information–based criteria (AIC, BIC), cross–validation (GCV), and maximum likelihood (ML) compare when selecting the smoothing parameter for the mean? Does the choice depend on the underlying signal or sample size?

2. *Scale selection and interval calibration.* Does the choice of criterion for the scale, and the way predictive intervals are calibrated, affect coverage accuracy and interval length? Which adjustments yield intervals with nominal 95 % coverage?

3. *Sample–size effects.* How quickly do mean and scale estimates converge as the number of observations increases? Do the conclusions regarding criteria and interval calibration change with sample size?

4. *Basis choice.* Is a truncated–power spline basis advantageous over the default B–spline basis, especially for small samples? How sensitive are the results to this design decision?

These questions map directly onto the tasks faced by practitioners: choosing a criterion for each block, deciding whether to calibrate predictive bands, anticipating data requirements, and selecting a basis. Throughout, we use insights from the theoretical development (smoothness–variance trade–offs in subsection 2.1) and the implementation details (RS alternation and diagnostics in section 3) to interpret the empirical patterns.

4.1.1 Data–generating algorithms (DGAs)

We generate covariates x independently and uniformly on $[0, 1]$, sorted for plotting. Given a scenario index DGA $\in \{\mathsf{smooth}, \mathsf{piecewise}, \mathsf{sine}\}$, the mean $\mu(x)$ and standard deviation $\sigma(x)$ are defined in Table 1. Each response is $y_i = \mu(x_i) + \varepsilon_i$ with independent $\varepsilon_i \sim \mathcal{N}(0, \sigma^2(x_i))$. The three DGAs cover progressively more challenging mean and variance structures: a smooth quadratic trend with parabolic heteroscedasticity (smooth), a piecewise linear mean with a variance jump (piecewise), and a sine wave with periodic variance (sine). The functional forms are succinctly summarized below and are reused in subsection 4.2 when interpreting the outcomes.

Table 1: Overview of the data–generating algorithms and experimental factors. Each scenario uses cubic splines ($\ell = 3$) with $m = 20$ basis functions and a second–order difference penalty. The noise is Gaussian with standard deviation $\sigma(x)$; heteroscedasticity patterns are indicated in the last column. The rightmost column lists the factors varied in our design: sample size n, mean and scale criteria, basis type (`tp_flag`) and refinement flag.

Scenario	$\mu(x)$		$\sigma(x)$	Heteroscedasticity
smooth	$2\,(x - \frac{1}{2})^2 - 0.1$		$0.3 + 0.2\,(x - \frac{1}{2})^2$	Parabolic (symmetric)
piecewise	$\begin{cases} -0.2 + 0.5\,x, & x < 0.4, \\ 0.1, & 0.4 \le x < 0.7, \\ 0.1 + 0.8\,(x - 0.7), & x \ge 0.7, \end{cases}$		$0.3 + 0.2\,(x - \frac{1}{2})^2 + 0.2\,\mathbf{1}_{\{x>0.6\}}$	Piecewise jump
sine	$0.3\,\sin(2\pi x)$		$0.3 + 0.2\,(x - \frac{1}{2})^2 + 0.1\,\sin(2\pi x)$	Periodic

Abbreviations: AIC = Akaike information criterion; BIC = Bayesian information criterion; GCV = generalized cross–validation; ML = maximum likelihood. The indicator $\mathbf{1}_{\{x>0.6\}}$ equals 1 when $x > 0.6$ and 0 otherwise.

For all DGAs, the spline degree is fixed at $\ell = 3$ (cubic), consistent with the default recommendations in section 3, and the number of interior knots is set to $m = 20$. The optional truncated–power basis (`tp_flag` = `TRUE`) replaces the default B–spline basis (`FALSE`); both bases share the same degree and knot locations. Smoothing parameters for the mean (λ_μ) and scale (λ_σ) are selected independently on a logarithmic grid. We consider four criteria for the mean and two for the scale, leading to eight criterion combinations.

4.1.2 Methods and safety mechanisms

Estimation proceeds via the RS alternation algorithm described in section 3. At each iteration, a penalized weighted least squares problem is solved for the mean (location) and then for the log–scale, with smoothing parameters chosen from a pre–specified grid by minimizing or maximizing the selected criterion. The key implementation details, distilled from the simulation driver script, are as follows:

- **Basis and penalty.** We use cubic splines ($\ell = 3$) with $m = 20$ interior knots and a second–order difference penalty ($r_{\text{pen}} = 2$) for both blocks. Knots are placed equidistantly over $[0, 1]$ with a $5\,\%$ buffer at each end. The basis can be either B–spline or truncated–power, toggled by `tp_flag`. This choice does not affect the penalty order but does change the representation of the null space; see subsection 2.1 for theoretical details.

- **Smoothing grids.** The base grid for each block contains 60 logarithmically spaced values between 10^{-4} and 10^3. Optionally, a refined grid of 120 values from 5×10^{-5} to 5×10^3 is scanned when `refine = TRUE`, allowing a fine adjustment around the preliminary optimum. Independent grids enable different degrees of smoothness for the mean and scale, an important feature for heteroscedastic data.

- **Selection criteria.** Only four criteria are allowed for the mean (AIC, BIC, GCV and ML) and two for the scale (GCV and ML). These choices reflect a trade–off between model fit and smoothness; the smoothness–variance interplay is discussed in subsection 2.1. BIC and AIC penalize complexity differently, GCV approximates leave–one–out cross–validation(Craven & Wahba, 1979), and ML maximizes the likelihood.

- **Refinement stage.** When `refine = TRUE`, a second sweep over the refined grid is invoked after the first iteration. In our runs this had little impact on the final fits; we report results pooled across the `refine` flag and return to this point in subsubsection 4.2.3.

- **Fallback backend.** If the package fails (e.g., due to numerical singularities or non–finite estimates), a fallback routine approximates the fit using fixed spline bases and GCV selection. This safeguard is activated rarely and produces a valid, albeit potentially coarser, fit. The simulation output records the backend (`pkg` or `fallback`) and stage (`pkg_base`, `pkg_refined`, `fallback`) for reproducibility.

- **Chunked execution and resume.** Each design cell (combination of DGA, n, criteria, basis and refinement flag) is replicated 80 times. Results are written incrementally to per–chunk CSV files, enabling the script to resume from interruptions. Seeds are set deterministically as `seed` + `id`, with `seed` fixed at 123 and `id` the row index in the design grid. This design ensures reproducible replicates and robust recovery if computing resources are limited.

These safety mechanisms complement the numerical stabilizers described in section 3 (ridge adjustments, Hutchinson trace estimator, etc.). In summary, the simulation pipeline mirrors the practical workflow advocated in Chapter 3: independent smoothing grids per block, cubic splines with second–order penalties, and automatic calibration of predictive intervals.

4.1.3 From design to inference

The experimental design couples the DGAs in Table 1 with the method choices above. Each cell is defined by a DGA, a sample size n, a basis flag (`tp_flag`), a pair of selection criteria $(\mathrm{crit}_\mu, \mathrm{crit}_\sigma)$, and a refinement indicator. We fit 80 replicates per cell and record the fitted mean $\hat{\mu}(x_i)$, the fitted scale $\hat{\sigma}(x_i)$ and various diagnostics. Our performance measures include:

- **Mean accuracy.** The root mean squared error for the mean, $\mathrm{RMSE}_\mu = \left(n^{-1}\sum_{i=1}^{n}(\hat{\mu}(x_i) - \mu(x_i))^2\right)^{1/2}$, averaged over replicates. Lower values indicate closer recovery of the true mean.
- **Interval coverage and length.** We form predictive intervals $L_i = \hat{\mu}(x_i) - z_{0.975}\,\hat{\sigma}(x_i)$ and $U_i = \hat{\mu}(x_i) + z_{0.975}\,\hat{\sigma}(x_i)$ and compute the empirical coverage $\mathbb{P}\{y_i \in [L_i, U_i]\}$. Because the raw intervals are too narrow, two adjustments are considered: leverage inflation and calibration via a multiplicative factor. The calibrated coverage (`coverage_cal`) is the primary target; see subsubsection 4.2.2 for details. We also record the average length of the intervals.
- **Smoothness.** Effective degrees of freedom (EDF) for the mean (edf_μ) and scale (edf_σ) quantify the flexibility chosen by each criterion.
- **Computation time.** The elapsed time per replicate informs the computational cost of each criterion combination.

With the design and metrics in place, we now turn to the results organised by the guiding questions.

4.2 Results

This section synthesises the aggregated results. For each research question, we present the key numerical trends, interpret them in light of the theoretical framework, and refer to the appropriate figures. All figures are included at the end of this chapter and are referenced by label (e.g., Figure 2).

4.2.1 Mean–criterion comparison

To address our first guiding question on mean selection, Figure 2 contrasts the average RMSE_μ across sample sizes and mean criteria for each DGA. Two general patterns emerge. First, errors decrease sharply as n increases: for the smooth DGA, the mean error drops from approximately 0.26 at $n = 200$ to 0.03 at $n = 1000$, and analogous reductions occur for the other DGAs. This aligns with the smoothness–variance trade–off discussed in subsection 2.1; more data allow stronger penalisation without sacrificing fidelity. Second, BIC and AIC consistently yield the lowest RMSE across all sample sizes and DGAs. For example, under the piecewise DGA at $n = 200$, BIC attains an average RMSE of 0.41 compared with 0.59 for ML and 0.27 for GCV; by $n = 1000$ the differences shrink but remain evident. GCV approaches AIC and BIC as n grows, whereas ML frequently overfits, producing large EDF (around 20) and high errors. The effective degrees of freedom in Figure 3 corroborate this interpretation: BIC typically selects around 5–6 degrees of freedom when $n \geq 500$, AIC and GCV select about 7, while ML uses nearly all 20 basis functions. These findings echo the default recommendations in section 3 and suggest that AIC or BIC should be preferred for the mean, with GCV a competitive alternative at larger sample sizes. ML is dominated in both accuracy and smoothness and offers no practical advantage.

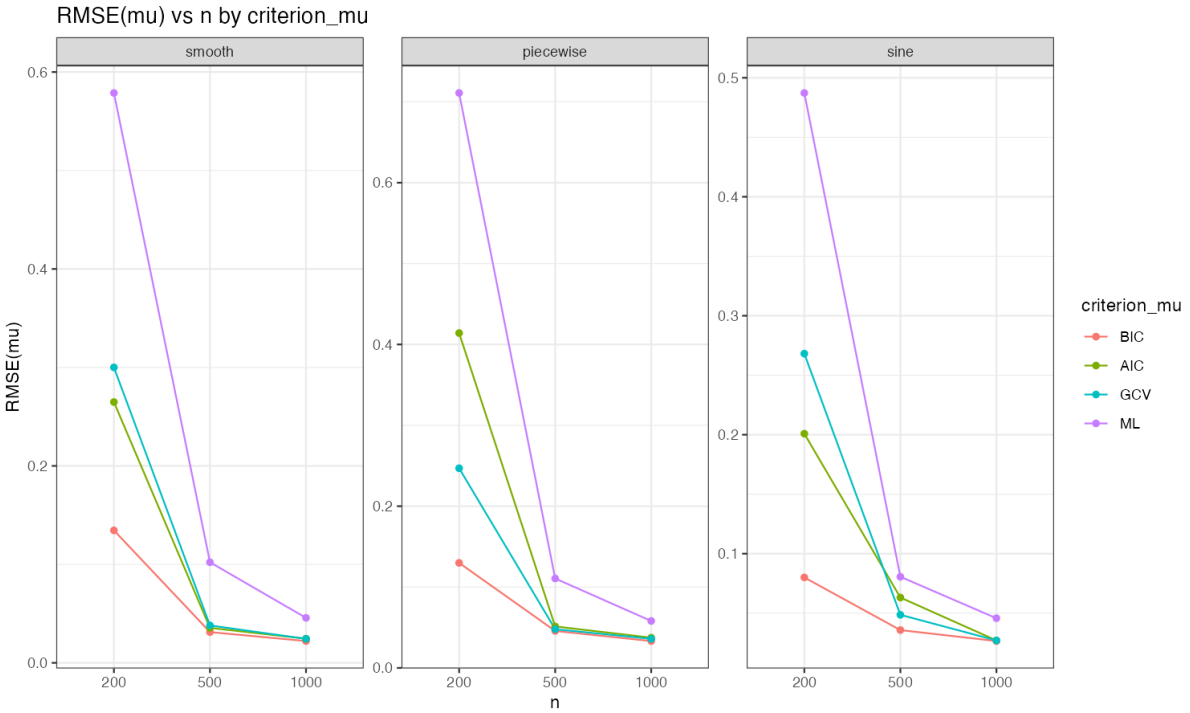

Figure 2: Mean root mean squared error (RMSE_μ) across sample sizes and mean criteria. Each panel corresponds to one DGA. Lines connect the mean RMSE over 80 replicates for each criterion; lower values are better.

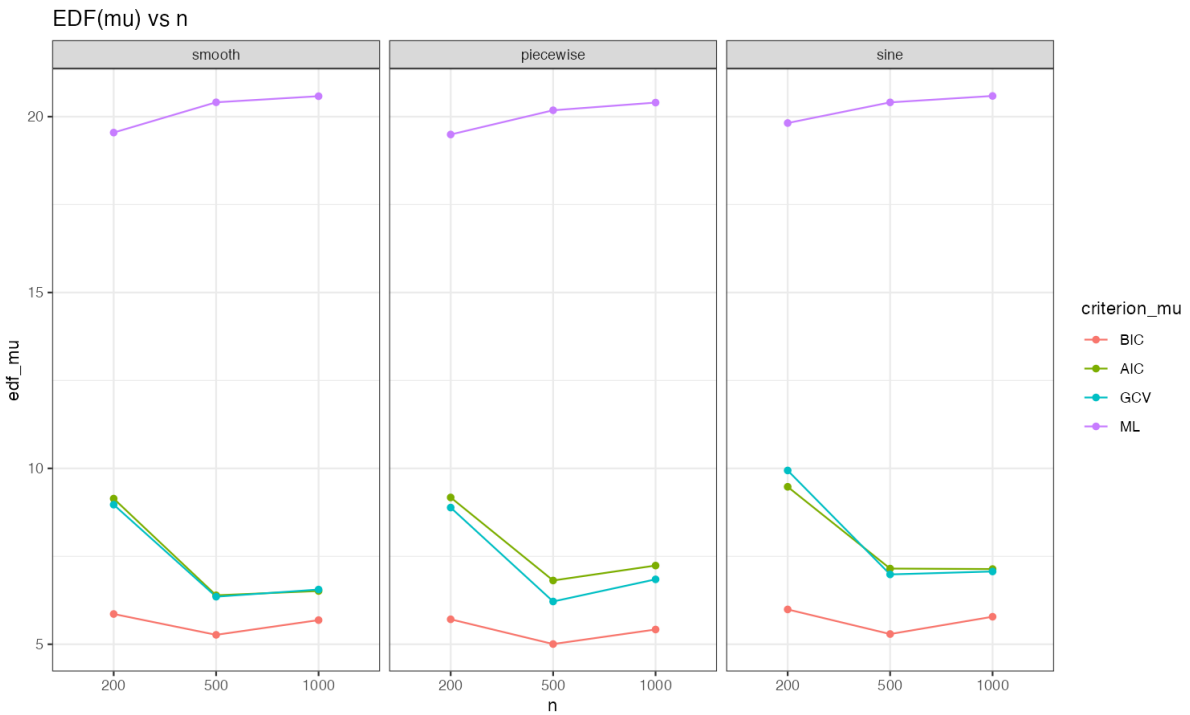

Figure 3: Effective degrees of freedom for the mean smoother across sample sizes and criteria. BIC selects the fewest degrees of freedom, followed by AIC and GCV; ML selects nearly all available basis functions.

4.2.2 Scale criterion and predictive interval calibration

Turning to our second guiding question concerning the scale criterion and interval calibration, predictive intervals derive from the scale fit and require careful calibration. Figure 4 reports the empirical coverage under three interval definitions—raw, leverage–inflated and calibrated—and two scale criteria (GCV and ML). The uncalibrated intervals under–cover severely: coverage averages around 0.72 for all DGAs and sample sizes. Inflating by the leverage factor improves coverage only marginally. In contrast, multiplying $\hat{\sigma}$ by a single calibration constant yields coverage close to the nominal 0.95 across all conditions. This outcome has a simple justification: raw intervals use $\hat{\sigma}$ directly, which underestimates the true variability because smoothing shrinks residuals; the calibration adjusts $\hat{\sigma}$ so that $\mathbb{P}\{Y \in [L, U]\} \approx 0.95$ by construction, exploiting the assumption of Gaussian errors. Consequently, we report `coverage_cal` as the primary metric. Average interval lengths are shown in Figure 5; calibrated intervals are approximately twice as long as raw intervals at $n = 200$ but shrink as n increases. Scale–criterion differences are minor: both GCV and ML achieve the same calibrated coverage, though ML selects substantially larger EDF for the scale (Figure 6) and incurs longer computation times (Figure 7). Given this, GCV is preferable for the scale block.

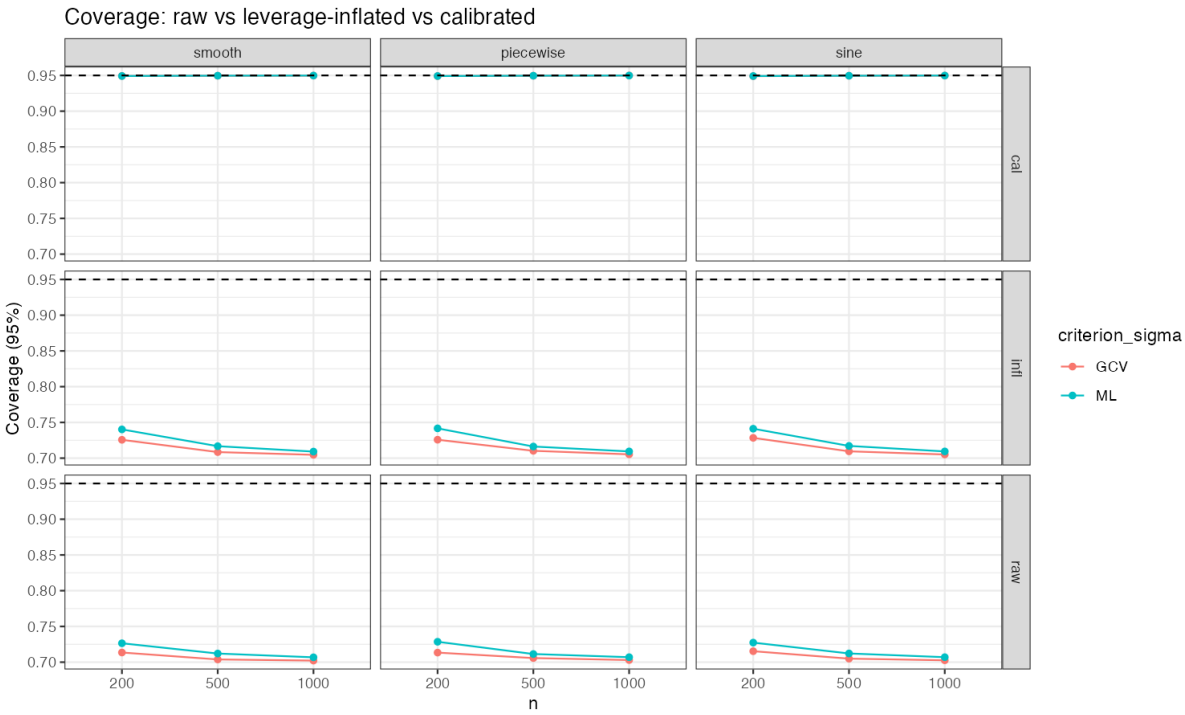

Figure 4: Empirical coverage of predictive intervals under different adjustments (rows) and scale criteria (colours). Nominal 95 % coverage is indicated by the dashed line. Calibration restores coverage to the nominal level across all DGAs and sample sizes; raw and leverage–inflated intervals consistently under–cover.

4.2.3 Sample–size effects

Addressing our third question about sample–size effects, we find that across all DGAs and method combinations, increasing the sample size improves both point and interval estimation. Mean RMSE declines nearly quadratically with n: halving n from 1000 to 500 roughly doubles the error, while halving again to 200 increases the error by another factor of four. Calibrated interval lengths also shrink with n (see Figure 5). The effective degrees of freedom chosen by AIC, BIC and GCV decrease modestly as n increases (Figure 3), reflecting stronger penalisation

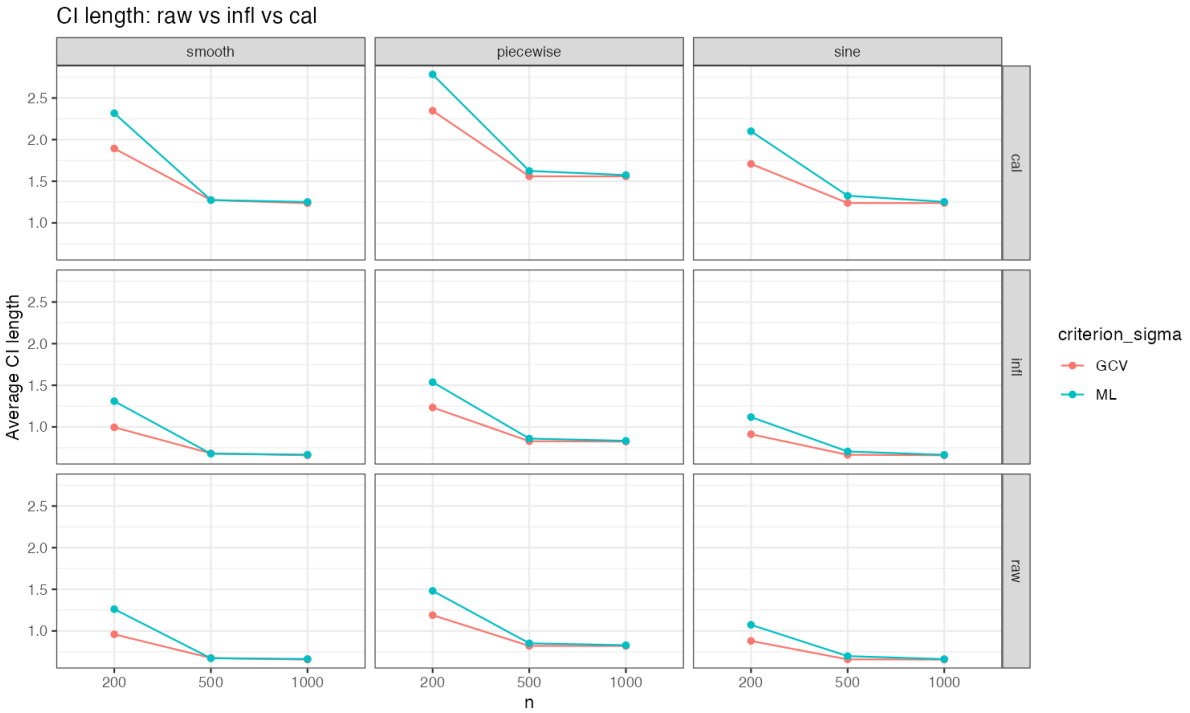

Figure 5: Average length of predictive intervals under different adjustments and scale criteria. Calibrated intervals are longer than raw intervals but narrow as n increases. Differences between GCV and ML for the scale are small.

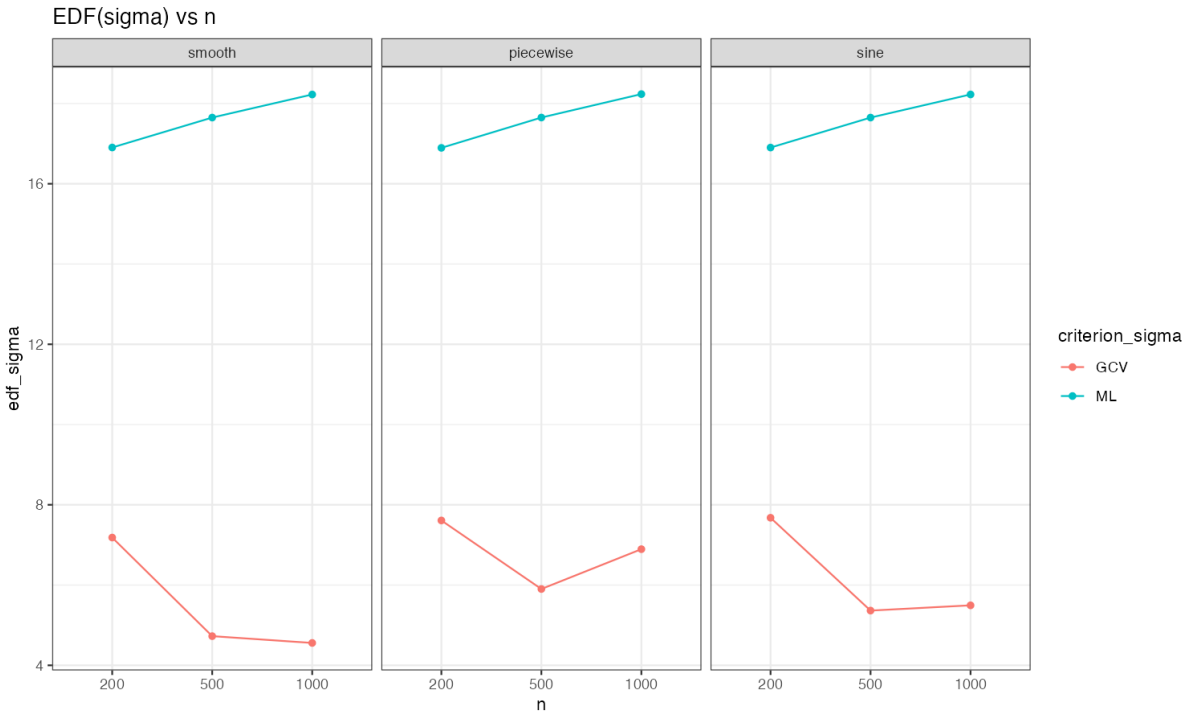

Figure 6: Effective degrees of freedom for the scale smoother. GCV selects approximately 7 degrees of freedom at $n = 200$ and stabilises around 5–6 at larger n. ML uses 17–18 degrees of freedom, leading to longer computation times without improving coverage or interval length.

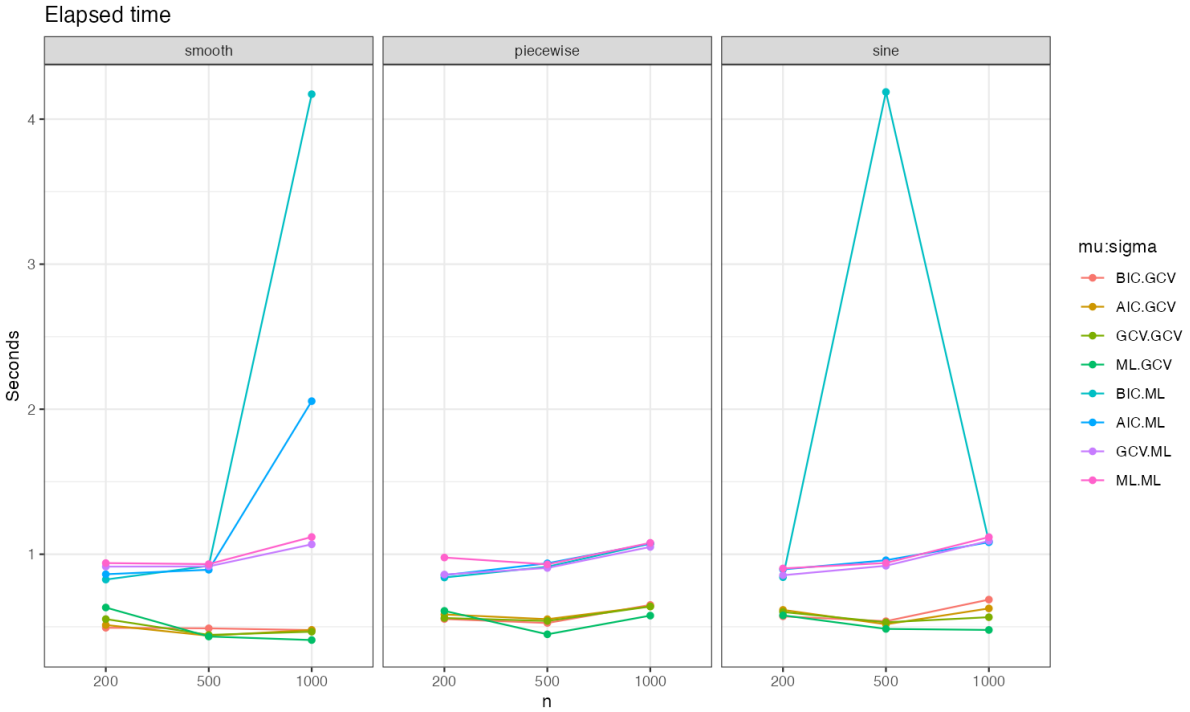

Figure 7: Average computation time per replicate across sample sizes, mean and scale criteria, and DGAs. Runs with ML for the scale (right column) take substantially longer, especially at $n = 500$ and $n = 1000$.

when more data are available. The optional refinement stage has negligible impact on these trends: the base grid is sufficiently dense, and the refined sweep rarely changes the selected λ. In practice, refinement can be disabled to save time. These results confirm the theoretical expectation that smoothing estimators converge at a rate depending on the roughness of the true function (cf. subsection 2.1) and reinforce the need for adequate sample size when estimating complex mean and variance structures.

4.2.4 Basis choice and the truncated–power flag

Finally, to address our fourth guiding question on the basis choice, we compare the B–spline and truncated–power bases. Figure 8 plots RMSE_μ by basis type. When $n = 200$, the B–spline basis (`tp_flag` = `FALSE`) yields RMSE values above 0.5 for all DGAs, with poor fits under the piecewise and smooth DGAs. Switching to the truncated–power basis (`tp_flag` = `TRUE`) reduces the error to around 0.12 on the smooth DGA, 0.07 on the sine DGA, and 0.12 on the piecewise DGA. As the sample size grows to 500 and 1000, the difference narrows: both bases attain RMSE below 0.08 at $n = 500$ and below 0.05 at $n = 1000$, with a slight advantage for the truncated–power basis. These observations suggest that truncated–power splines are more stable when data are scarce, likely because they allocate a separate polynomial part and have localised influence in the truncated terms, reducing boundary bias. This behaviour is consistent with the theoretical discussion of bases and penalties in subsection 2.1 and section 3. We therefore recommend enabling `tp_flag = TRUE` when $n \lesssim 500$; for larger samples, either basis suffices.

4.3 Practical recommendations

The simulation results yield the following actionable guidelines for users of `asp25psplines`:

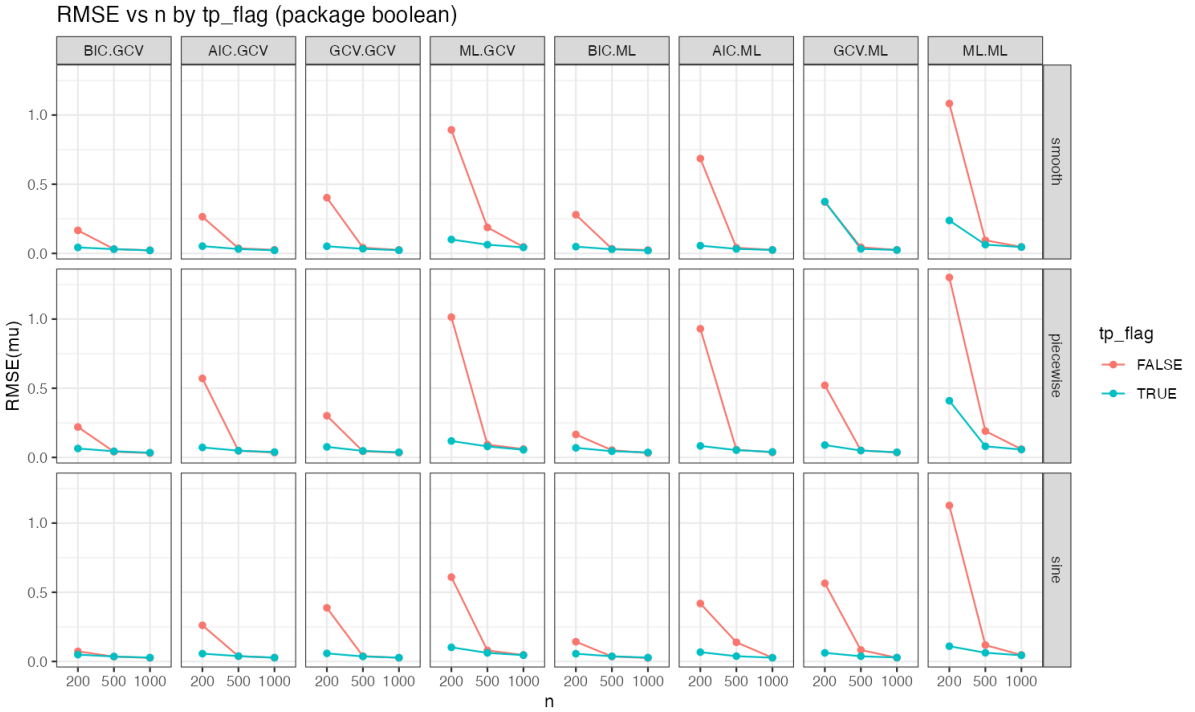

Figure 8: Mean RMSE stratified by basis type (`tp_flag`). Each panel corresponds to one DGA, and lines connect the mean over all criterion combinations and 80 replicates. At $n = 200$ the truncated–power basis reduces RMSE by an order of magnitude compared with the B–spline basis; differences diminish at larger n.

- **Use cubic splines with moderate basis size.** Twenty basis functions and a second–order penalty strike a good balance between flexibility and stability. Independent smoothing grids per block are essential to decouple the mean and scale fits.
- **Select AIC or BIC for the mean.** Both criteria achieve low errors with moderate EDF. GCV can be used when the sample size is large; ML should be avoided because it overfits and inflates EDF.
- **Select GCV for the scale and calibrate intervals.** The scale criterion has little effect on coverage once calibration is applied, but GCV is computationally cheaper than ML. Always compute calibrated predictive intervals; uncalibrated intervals under–cover dramatically.
- **Enable the truncated–power basis when $n \leq 500$.** This option stabilises small–sample fits without measurable cost. For larger samples, both bases perform similarly.
- **Refinement is optional.** Scanning a refined λ grid rarely changes the selected smoothing parameters; the base grid is adequate for routine analysis.

4.4 Reproducibility and computational aspects

To reproduce the simulation, load the `asp25psplines` package and ensure the functions described in section 3 are available on the library path. The driver script sources the package functions, constructs the design grid in Table 1, and iterates through the cells. Each replicate is seeded deterministically as described in subsubsection 4.1.2. Intermediate results are written to CSV files chunk by chunk; the final aggregated results and figures used in this chapter are regenerated by running the script end–to–end. The simulation was conducted on a single core;

typical runtimes range from around 0.5 seconds per fit when $\text{crit}_\sigma = \text{GCV}$ to 1–1.5 seconds when $\text{crit}_\sigma = \text{ML}$ (see Figure 7). All software versions, random seeds and knot settings are stored with the results to ensure exact replication. Detailed instructions for installing dependencies and running the script are provided in the supplementary materials.

In summary, this chapter demonstrates that the `asp25psplines` estimator recovers complex mean and scale functions reliably, provided that appropriate smoothing–parameter criteria and bases are selected. The empirical patterns corroborate the theoretical properties discussed in the earlier chapters and yield concrete recommendations for applied analysis.

5 Conclusion

This work develops and evaluates a penalized spline approach to heteroskedastic location–scale regression. Motivated by the ubiquity of nonlinear trends and nonconstant variance in empirical data, we formulated a Gaussian model in which both the conditional mean and the conditional standard deviation are smooth functions of a single covariate. Chapter 2 laid the foundation for this framework. Building on generalized additive models, we showed how cubic spline bases with difference penalties provide flexible yet parsimonious representations of unknown smooth functions. The penalty's difference order controls curvature while preserving a low–order polynomial null space, and the choice between B–spline and truncated–power bases determines how this null space interacts with the penalty. We derived penalized weighted least squares problems for the mean and log–scale, introduced effective degrees of freedom via smoother matrices, and described an Rigby–Stasinopoulos algorithm for generalized additive models for location, scale and shape (GAMLSS) that iteratively updates the two blocks until the global deviance stabilizes. Theorems in Chapter 2 guarantee that this scheme is well defined and converge to a stationary point of the penalized likelihood. Independent smoothing parameters for the mean and scale play a crucial role: they allow the mean to be smoother or rougher than the scale depending on the data, and they are selected via information criteria or cross–validation on coarse–to–fine grids.

Chapter 3 translated these ideas into a robust R implementation, `asp25psplines`. The package exposes configurable choices—basis type, knot placement, penalty order, smoothing grids and criteria—while encapsulating numerical safeguards such as ridge adjustments, Hutchinson trace approximation for effective degrees of freedom and QR fallback solvers. It implements the RS alternation described above and provides functions for fitting, prediction, visual diagnostics and summary tables. Both the design matrices and knot settings are stored with the fitted object so that predictions at new covariates reuse the same basis. Default settings follow the recommendations derived from theory: cubic splines, second–order penalties, equidistant knots with a small boundary buffer, and independent smoothing grids per block. The user can override these defaults when data structure suggests alternative choices.

Chapter 4 investigated the empirical performance of the estimator through a comprehensive simulation study. We designed three data–generating algorithms—smooth, piecewise and sine—that vary the complexity of the mean and variance, and crossed them with multiple sample sizes, criterion pairs, basis types and an optional refinement sweep. Each cell was replicated 80 times, and we recorded root mean squared error for the mean, predictive interval coverage and length, effective degrees of freedom and computation time. The results answer the four guiding questions posed in subsection 4.1: **(i)** *mean selection*: AIC and BIC consistently yield the lowest mean errors and moderate degrees of freedom, whereas maximum likelihood overfits, and generalized cross–validation becomes competitive only in larger samples; **(ii)** *scale selection and calibration*: once predictive intervals are calibrated by scaling the estimated standard deviation, both generalized cross–validation and likelihood achieve nominal coverage, but raw and leverage–inflated intervals under–cover severely; **(iii)** *sample size*: estimation accuracy and interval width improve markedly as the number of observations increases, and the optional refinement of the smoothing grid has negligible impact; and **(iv)** *basis choice*: a truncated–power basis dramatically reduces error when the sample size is small, with differences diminishing as data grow. These findings reinforce the theoretical message that separate smoothing parameters are necessary for the mean and scale, that shrinkage and calibration must be tuned jointly, and that the choice of basis can matter in finite samples.

Putting these strands together, our study offers several practical lessons. For heteroskedastic location–scale regression with one covariate, we recommend using cubic P–splines with around 20 interior knots and a second–order difference penalty, selecting smoothing parameters independently by AIC or BIC for the mean and by generalized cross–validation for the scale, and

calibrating predictive intervals via a multiplicative adjustment to the estimated standard deviation. When the sample size is modest (roughly fewer than 500 observations), a truncated–power basis yields more stable fits and should be preferred. The optional refinement of the smoothing grid is computationally expensive and rarely alters the selected parameters. These guidelines, grounded in both theory and simulation, can serve as defaults for practitioners. We note that our implementation is robust to numerical failures via a fallback routine and a chunked execution scheme, ensuring reproducibility even on limited computational resources.

There are several avenues for further research. Extending the framework to multiple covariates would entail tensor–product bases and careful treatment of interactions; generalization beyond Gaussian responses would require link functions and distributional assumptions in the spirit of GAMLSS. Alternative methods for smoothing–parameter selection, such as restricted maximum likelihood or Bayesian priors on the coefficients, merit exploration. The calibration of predictive bands could be refined by deriving simultaneous confidence envelopes or by incorporating bootstrap techniques to better capture finite–sample variability. A natural next step—left unfinished here—would be an extended `predict()` that, for new covariates, returns mean and scale estimates together with calibrated predictive distributions and both pointwise and simultaneous intervals that propagate uncertainty from smoothing–parameter selection and spline estimation; despite promising prototypes, we did not obtain a numerically stable and computationally efficient implementation suitable for release. Finally, empirical analyses on real data would demonstrate the utility of our approach in substantive applications such as environmental monitoring, finance, and biomedical studies.

In conclusion, penalized spline location–scale regression provides a flexible and interpretable tool for modeling nonlinear signals and heteroskedastic variability. Our theoretical formulation, implementation and simulation evidence combine to show that careful choice of basis, penalty and smoothing criteria yields accurate estimators and well–calibrated predictive intervals. The `asp25psplines` package delivers these tools to users in a coherent interface. We hope that this work encourages wider adoption of location–scale smoothing methods and sparks further methodological developments in this fertile area.

Package ‘asp25psplines’

September 15, 2025

Title Penalised P-Splines for Gaussian Location-Scale Models

Version 0.1.0

Description 'asp25psplines' implements penalised B-spline regression for univariate Gaussian location-scale models. The conditional mean and the log-scale are represented by smooth functions built from B-spline bases with difference penalties (P-splines). The package provides modular building blocks (knot placement, basis construction, penalty matrices), an alternating fitting scheme for location and scale via penalised weighted least squares, and a high-level wrapper for automatic smoothing parameter selection and optional interval calibration. Convenience helpers are included for prediction, plotting, and concise summaries.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1)

Imports stats,
graphics,
grDevices

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Contents

asp25psplines-package 1
base_fun 2
calc_deviance 3
calc_mu 3
calc_sigma 4
create_knots 5
fit_spline 6
get_initial_values 7

get_pen_mat 8
plot_all 8
run_all_funcs 9
run_demo 10
summary_all 11

`asp25psplines-package` *asp25psplines: Penalised P-splines for Gaussian Location-Scale Models*

Description

Provides modular building blocks (knots, bases, penalties) and a high-level wrapper to alternate mean and log-scale estimation with optional calibration. See the help topics of the exported functions for usage.

Author(s)

Maintainer: Moritz Dallmann <moritz.dallmann@stud.uni-goettingen.de>

Authors:

- Nicolas Wackermann <nicolas.wackermann@stud.uni-goettingen.de>

`base_fun` *Base B-spline construction utility*

Description

Low-level helper to construct B-spline basis functions.

Usage

```
base_fun(x, knots, m, j)
```

Arguments

`x`	Numeric vector of covariates.
`knots`	List with elements `knots` (numeric knot vector), `tp` (logical thin-plate flag), and `l` (integer degree/order).
`m`	Integer penalty/order parameter (difference order).
`j`	Integer basis index.

Value

Numeric vector with basis evaluations for index `j`.

Examples

```
x <- seq(0, 1, length.out = 10)
K <- base_fun(
  x,
  knots = list(knots = seq(0, 1, length.out = 5), tp = FALSE, l = 2),
  m = 2, j = 1
)
```

---

calc_deviance *Calculate deviance for Gaussian location-scale model*

Description

Calculate deviance for Gaussian location-scale model

Usage

```
calc_deviance(y, mu_hat, sigma_hat)
```

Arguments

y	Numeric response vector.
mu_hat	Numeric vector of fitted means.
sigma_hat	Numeric vector of fitted standard deviations.

Value

Numeric deviance-like scalar (or vector) for the fit.

Examples

```
y <- rnorm(10); mu <- rep(0,10); sig <- rep(1,10)
calc_deviance(y, mu, sig)
```

`calc_mu` *Update step for the mean*

Description

Penalised weighted least squares update for the mean curve.

Usage

```
calc_mu(
  X,
  y,
  K_mu,
  sigma_hat,
  lambda_grid,
  method = "AIC",
  max_iterations_mu = 50,
  tolerance = 1e-06
)
```

Arguments

`X`	Design matrix for the mean.
`y`	Numeric response vector.
`method`	Character; selection method for smoothing (e.g., "GCV","BIC").
`tolerance`	Numeric convergence tolerance.
`K\_mu`	Penalty matrix for the mean.
`sigma\_hat`	Current estimate of the standard deviation (vector).
`lambda\_grid`	Numeric vector or scalar of smoothing values.
`max\_iterations\_mu`	Integer; max iterations for the mean step.

Value

A list with updated coefficients, fitted values and selected lambda.

Examples

```
# pseudo:
# out <- calc_mu(X, y, K_mu, sigma_hat, lambda_grid=10^seq(-3,3), method="GCV",
#                max_iterations_mu=50, tolerance=1e-6)
```

calc_sigma — *Update step for the log-scale*

Description

Penalised weighted least squares update for the log-scale.

Usage

```
calc_sigma(
  Z,
  y,
  K_sigma,
  mu_hat,
  lambda_grid,
  method = "AIC",
  max_iterations_sigma = 50,
  tolerance = 1e-06
)
```

Arguments

`Z`	Design matrix for the log-scale.
`y`	Numeric response vector.
`method`	Character; selection method for smoothing (e.g., "GCV","BIC").
`tolerance`	Numeric convergence tolerance.
`K\_sigma`	Penalty matrix for the scale.
`mu\_hat`	Current estimate of the mean (vector).
`lambda\_grid`	Numeric vector or scalar of smoothing values.
`max\_iterations\_sigma`	Integer; max iterations for the scale step.

Value

A list with updated coefficients, fitted log-scale and selected lambda.

Examples

```
# pseudo:
# out <- calc_sigma(Z, y, K_sigma, mu_hat, 10^seq(-3,3), "GCV", 50, 1e-6)
```

`create_knots` *Create knot locations for P-spline basis construction*

Description

Create knot locations for P-spline basis construction

Usage

```
create_knots(
  x,
  a = min(x),
  b = max(x),
  m = 20,
  l = 3,
  equi = TRUE,
  tp = FALSE,
  buffer = 0.05
)
```

Arguments

`x`	Numeric covariate vector.
`a`	Numeric lower boundary of the covariate range (left endpoint of $[a, b]$).
`b`	Numeric upper boundary of the covariate range (right endpoint of $[a, b]$).
`m`	Integer penalty/order parameter for differences.
`l`	Integer number of interior knots or basis size control.
`equi`	Logical; if TRUE, equally spaced knots, otherwise quantile-based.
`tp`	Logical; if TRUE, use thin-plate style option if implemented.
`buffer`	Numeric; extra range added beyond $[a, b]$ for boundary knots.

Value

A numeric vector of knot locations.

Examples

```
create_knots(x = runif(50), a = 0, b = 1, m = 2, l = 10, equi = TRUE, tp = FALSE, buffer = 0.05)
```

fit_spline — *Fit a penalised B-spline smoother*

Description

Fit a penalised B-spline smoother

Usage

```
fit_spline(x, knots)
```

Arguments

x	Numeric covariate vector or design input.
knots	Numeric vector of knot locations.

Value

A list with basis/design matrix and fitted coefficients.

Examples

```
x <- seq(0, 1, length.out = 50)
k <- list(
  knots = seq(0, 1, length.out = 10),
  tp    = FALSE,
  l     = 2
)
fs <- fit_spline(x = x, knots = k)
```

get_initial_values — *Initial values for location-scale fitting*

Description

Initial values for location-scale fitting

Usage

```
get_initial_values(X, Z, y)
```

Arguments

X	Design matrix for the mean.
Z	Design matrix for the log-scale.
y	Numeric response vector.

Value

A list with initial mean and scale (or their coefficients).

Examples

```
# init <- get_initial_values(X, Z, y)
```

---

get_pen_mat — *Create Penalty Matrix for respective spline method*

Description

For B-splines a discrete difference penalty of order `r` is used. For truncated power splines the unpenalised part is the polynomial of degree 0..l; the truncated parts (one per inner knot) are penalised with an identity penalty.

Usage

```
get_pen_mat(knots, r = 2)
```

Arguments

`knots`	A list as returned by `create\_knots()` with entries `knots`, `tp`, and `l`.
`r`	Integer order of differences for P-splines (default 2).

Value

A symmetric positive semi-definite penalty matrix.

---

plot_all — *Quick diagnostic plot for fitted mean and (optionally) scale*

Description

Quick diagnostic plot for fitted mean and (optionally) scale

Usage

```
plot_all(result, x, y, alpha = 0.05, show_sigma = TRUE)
```

Arguments

result	Fitted object returned by run_all_funcs().
x	Optional numeric x-values for plotting on a specific grid.
y	Optional observed responses for residual diagnostics.
alpha	Numeric alpha level for bands (0–1). Default: 0.1.
show_sigma	Logical; if TRUE, draw a band derived from the estimated scale.

Value

Invisibly returns NULL. Called for its side effects (plot).

Examples

```
# res <- run_all_funcs(x, y)
# plot_all(res, show_sigma = TRUE)
```

---

run_all_funcs *High-level fit for Gaussian location–scale P-splines*

Description

Fits the conditional mean and log-scale by alternating penalised weighted least squares. Smoothing parameters are selected per component; an optional refinement step can calibrate interval coverage.

Usage

```
run_all_funcs(
  x,
  y,
  m = 20,
  l = 3,
  equi = TRUE,
  tp = FALSE,
  buffer = 0.05,
  r_pen = 2,
  lambda_grid_mu = exp(seq(log(1e-04), log(10000), length.out = 40)),
  lambda_grid_sigma = exp(seq(log(1e-04), log(10000), length.out = 40)),
  method_mu = "BIC",
  method_sigma = "BIC",
  max_iterations = 50,
  tolerance = 1e-05
)
```

Arguments

`x`	Numeric covariate vector.
`y`	Numeric response vector of the same length as `x`.
`m`	Integer difference order (penalty).
`l`	Integer basis size/degree controller (as used in your implementation).
`equi`	Logical; if TRUE, uses equally spaced knots, otherwise quantile-based.
`tp`	Logical; thin-plate style option if implemented.
`buffer`	Numeric boundary buffer beyond $[a, b]$ for boundary knots.
`r_pen`	Numeric or matrix; additional penalty specification if applicable.
`lambda_grid_mu`	Numeric vector of candidate smoothing values for the mean.
`lambda_grid_sigma`	Numeric vector of candidate smoothing values for the scale.
`method_mu`	Character; selection criterion for `lambda_mu` (e.g., "GCV", "BIC").
`method_sigma`	Character; selection criterion for `lambda_sigma`.
`max_iterations`	Integer; maximum number of alternating iterations.
`tolerance`	Numeric convergence tolerance for the alternating updates.

Value

An object of class `asp25psplines_fit` with elements such as `mu_hat`, `sigma_hat`, `coef_mu`, `coef_sigma`, `lambda_mu`, `lambda_sigma`, and `diagnostics`.

Examples

```
set.seed(1)
x <- sort(runif(80)); y <- sin(2*pi*x) + rnorm(80, 0.25)
res <- run_all_funcs(x, y, m = 2, l = 10, equi = TRUE, tp = FALSE,
buffer = 0.05, lambda_grid_mu = NULL, lambda_grid_sigma = NULL,
method_mu = "GCV", method_sigma = "GCV", max_iterations = 20)
```

`run_demo` *Minimal demo for asp25psplines*

Description

Minimal demo for asp25psplines

Usage

```
run_demo(seed = 1)
```

Arguments

seed	Integer random seed for reproducibility.

Value

A fitted object (class asp25psplines_fit) from a tiny example.

Examples

```
run_demo(seed = 1)
```

---

summary_all — *Compact console summary for a fitted asp25psplines model*

Description

Compact console summary for a fitted asp25psplines model

Usage

```
summary_all(result)
```

Arguments

result	Fitted object returned by run_all_funcs().

Value

A list with key summary elements; prints a concise summary.

Examples

```
# res <- run_all_funcs(x, y)
# summary_all(res)
```

Bibliography

Statement on AI Assistance. We acknowledge that we used generative-AI tools (e.g., ChatGPT) as aditional aid for early project outlining, code refactoring, and descriptive help (for example, summarizing features of the R package). These tools were used for summarization and language polishing only. We did *not* delegate creative or scientific work—such as study design, data analysis, statistical modeling, original code development, or interpretation of results—to AI systems. All datasets, scripts, and results were produced and verified by the authors, and all citations were checked against primary sources. No confidential or personally identifying data were shared with AI systems. The authors accept full responsibility for the integrity and accuracy of the content.

H. Akaike. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control, 19*(6), 716–723. https://doi.org/10.1109/TAC.1974.1100705

R. Harald Baayen and M. Linke. (2020). *An introduction to the generalized additive model (GAM).* In *Generalized additive models for linguists.* University of Tübingen.

P. Breheny. (2013). *The IRLS algorithm for fitting generalized linear models.* Lecture notes, University of Iowa.

P. Craven and G. Wahba. (1979). Smoothing noisy data with spline functions: Estimating the correct degree of smoothing by the method of generalized cross-validation. *Numerische Mathematik, 31*, 377–403. https://doi.org/10.1007/BF01389674

P. H. C. Eilers and B. D. Marx. (1996). Flexible smoothing with B-splines and penalties. *Statistical Science, 11*(2), 89–102. https://doi.org/10.1214/ss/1038425655

M. F. Hutchinson. (1990). A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics–Simulation and Computation, 19*(2), 433–450.

K. Kagerer. (2013). *A short introduction to splines in least squares regression analysis* (Working Paper No. 472). University of Regensburg.

T. Lyche, C. Manni, and H. Speleers. (2017). B-splines and spline approximation. In *CIME lecture notes.*

R. A. Rigby and D. M. Stasinopoulos. (2005). Generalized additive models for location, scale and shape. *Journal of the Royal Statistical Society: Series C (Applied Statistics), 54*, 507–554. https://doi.org/10.1111/j.1467-9876.2005.00510.x

D. M. Stasinopoulos and R. A. Rigby. (2007). Generalized additive models for location, scale and shape (GAMLSS) in R. *Journal of Statistical Software, 23*(7), 1–46. https://doi.org/10.18637/jss.v023.i07

G. Schwarz. (1978). Estimating the dimension of a model. *The Annals of Statistics, 6*(2), 461–464. https://doi.org/10.1214/aos/1176344136

S. N. Wood. (2017). *Generalized additive models: An introduction with R* (2nd ed.). Chapman & Hall/CRC.

List of Figures

1 Flow of the standalone implementation. Penalised IRLS is used for both blocks; blockwise smoothing is selected by a criterion over a grid Λ (AIC/BIC/ML/GCV). Effective degrees of freedom are computed exactly when feasible or approximated via a Hutchinson trace estimator(Breheny, 2013; Hutchinson, 1990). 9

2 Mean root mean squared error (RMSE_μ) across sample sizes and mean criteria. Each panel corresponds to one DGA. Lines connect the mean RMSE over 80 replicates for each criterion; lower values are better. 14

3 Effective degrees of freedom for the mean smoother across sample sizes and criteria. BIC selects the fewest degrees of freedom, followed by AIC and GCV; ML selects nearly all available basis functions. 14

4 Empirical coverage of predictive intervals under different adjustments (rows) and scale criteria (colours). Nominal 95 % coverage is indicated by the dashed line. Calibration restores coverage to the nominal level across all DGAs and sample sizes; raw and leverage–inflated intervals consistently under–cover. 15

5 Average length of predictive intervals under different adjustments and scale criteria. Calibrated intervals are longer than raw intervals but narrow as n increases. Differences between GCV and ML for the scale are small. 16

6 Effective degrees of freedom for the scale smoother. GCV selects approximately 7 degrees of freedom at $n = 200$ and stabilises around 5–6 at larger n. ML uses 17–18 degrees of freedom, leading to longer computation times without improving coverage or interval length. 16

7 Average computation time per replicate across sample sizes, mean and scale criteria, and DGAs. Runs with ML for the scale (right column) take substantially longer, especially at $n = 500$ and $n = 1000$. 17

8 Mean RMSE stratified by basis type (`tp_flag`). Each panel corresponds to one DGA, and lines connect the mean over all criterion combinations and 80 replicates. At $n = 200$ the truncated–power basis reduces RMSE by an order of magnitude compared with the B–spline basis; differences diminish at larger n. . . 18

Declaration

I hereby declare that I have produced this work independently and without outside assistance, and have used only the sources and tools stated.
I have clearly identified the sources of any sections from other works that I have quoted or given in essence.
I have complied with the guidelines on good academic practice at the University of Göttingen. If a digital version has been submitted, it is identical to the written one. I am aware that failure to comply with these principles will result in the examination being graded “nicht bestanden”, i.e. failed.

September 15, 2025

Nicolas Wackermann

Moritz Dallmann